

1 Summary

2 Modern scientific research requires consistent document preparation, re-
3 producible computation, unified data management, and transparent figure
4 provenance—yet researchers rely on fragmented tools that increase cognitive
5 load and introduce irreproducibility. We present SciTeX, an AI-native, mod-
6 ular research automation platform providing four integrated modules: Writer
7 for structured LaTeX manuscript preparation with containerized builds; Scholar
8 for literature management with automated metadata extraction; Viz for
9 publication-quality figure generation with embedded provenance; and Code
10 for reproducible computation with GPU and container support. Each mod-
11 ule operates independently while sharing a unified project structure through
12 lightweight symlinks and global identifiers, enabling cross-module traceabil-
13 ity. SciTeX reduces the activation energy required for rigorous, reproducible
14 science while maintaining compatibility with traditional workflows. The plat-
15 form is open source and follows FAIR principles. This manuscript demon-
16 strates SciTeX’s self-descriptive capabilities by being prepared within the
17 platform it documents.

18 Highlights

- 19 • SciTeX integrates manuscript writing, figure generation, literature man-
20 agement, and computation in a unified platform
- 21 • Modular architecture enables independent use of each component while
22 providing synergy through integration
- 23 • Containerized compilation ensures reproducible manuscript output across
24 all operating systems
- 25 • Embedded metadata enables full provenance tracking from figures and
26 results back to generating code
- 27 • AI-native design provides integration points for LLM-assisted research
28 workflows

29 SciTeX: An AI-Native Modular Platform for
30 Reproducible Scientific Research

31 Yusuke Watanabe^{a,*}

32 ^a*SciTeX.ai, Tokyo, Japan*

33 *Keywords:* reproducible research, scientific workflow automation, data
34 science platform, AI-native tools, figure provenance, FAIR principles

35 ~ 8 figures, 3 tables, 136 words for abstract, and 3056 words for main
36 text

37 **1. Introduction**

38 Scientific workflows increasingly span multiple heterogeneous environ-
39 ments: personal laptops, HPC systems, cloud servers, and containerized
40 pipelines [1, 2]. Traditional manuscript preparation tools remain fragmented,
41 requiring manual management of figures, references, code execution, and ver-
42 sion control. These inconsistencies lead to irreproducible results, duplicated
43 effort, and high cognitive load that distracts researchers from their primary
44 scientific objectives [3].

45 The preparation of scientific manuscripts involves numerous technical
46 challenges that extend beyond the intellectual task of communicating re-
47 search findings. Researchers must navigate complex typesetting systems,
48 manage multiple document versions, coordinate figures and tables across for-
49 mats, and ensure reproducible compilation environments [4]. Furthermore,
50 maintaining consistency between computational analyses, their visual repre-
51 sentations, and the manuscript text requires constant manual synchronization—
52 a process prone to error and version drift [5].

53 Traditional approaches to manuscript preparation typically rely on lo-
54 cal LaTeX installations, where specific versions of packages and compilation

*Corresponding author. Email: yusuke.watanabe@scitex.ai

tools can vary significantly across machines and over time. This variability creates reproducibility challenges, particularly in collaborative environments where multiple authors work on different systems [? ?]. Similarly, figure generation often occurs in isolation from the manuscript, with researchers using various plotting libraries without standardized metadata or provenance tracking. Literature management frequently depends on external tools that operate independently from the writing environment, leading to citation inconsistencies and manual bibliography maintenance.

Existing solutions have addressed individual aspects of this problem. ~~Overleaf and similar cloud-based platforms~~ Cloud-based platforms such as Overleaf provide consistent compilation environments but require continuous internet connectivity and separate figure generation workflows. Zotero and Mendeley excel at citation management but lack integration with manuscript preparation and computational analysis. Jupyter notebooks and Binder enable reproducible computation but provide limited support for publication-quality figures or structured manuscript writing [?]. Manubot pioneered open collaborative writing with automated citation processing [?], while Pandoc Scholar demonstrated agile multi-format document generation [?]. SigmaPlot and Origin offer sophisticated visualization capabilities but operate as standalone applications without provenance tracking or manuscript integration.

The fundamental challenge lies in unifying these disparate tools while maintaining the flexibility that researchers require [?]. The solution must accommodate diverse content types, multiple output documents, and varying journal requirements while ensuring a single source of truth for shared elements. Additionally, modern research increasingly involves AI-assisted workflows for code generation, literature review, and manuscript revision—capabilities that require native integration rather than ad-hoc addition [? ?].

Version control systems, particularly Git, have emerged as powerful tools for managing research workflows [? ?], enabling transparent tracking of

changes and facilitating collaboration [? ?]. However, effectively combining Git with LaTeX, figure generation, and literature management requires significant technical expertise that many researchers lack [?].

Containerization technologies such as Docker have transformed software reproducibility [? ? ?], and several frameworks have emerged to leverage containers for reproducible research [? ?]. The Rockerverse ecosystem demonstrates how containers can support reproducible R-based workflows [?], while tools like CREDO simplify Docker configuration for bioinformatics [?]. End-to-end templates for reproducible analysis and manuscript writing have been proposed [? ?], yet comprehensive integration across all research activities remains elusive.

SciTeX addresses these challenges through a unified, modular, AI-native platform that allows researchers to: write manuscripts in a structured LaTeX environment with containerized compilation; manage citations and PDFs with consistency and automated metadata extraction; generate publication-quality figures with embedded provenance metadata; and execute analyses in controlled environments with synchronized results [?]. Importantly, SciTeX maintains bidirectional low migration cost: each module functions independently, providing value without requiring adoption of the entire ecosystem, while synergy across modules enables a strictly superior integrated workflow.

The platform follows principles articulated in seminal works on reproducible research [? ? ?] while incorporating modern infrastructure for containerized compilation [? ?], workflow automation [? ?], and FAIR data practices [?]. By embedding reproducibility infrastructure into everyday research tools, SciTeX makes rigorous practices the path of least resistance rather than an additional burden [? ?].

Many researchers lack unified tools to integrate writing, computation, visualization, and literature management within a single, reproducible environment. Graduate students and early-career researchers often spend disproportionate time learning fragmented toolchains rather than conducting research. Computational scientists require reproducible pipelines that connect analysis code to manuscript figures without manual intervention [?].

AI-augmented research workflows demand native integration points for automated assistance. SciTeX provides this missing infrastructure, reducing the activation energy required for rigorous, reproducible science while democratizing access to advanced computational workflows for researchers unfamiliar with Linux, Git, or containers.

This manuscript is prepared within SciTeX Writer, referencing figures generated in SciTeX Viz, bibliography managed in SciTeX Scholar, and computational environments controlled by SciTeX Code—thereby making the manuscript a live demonstration of the platform’s integrated capabilities.

2. Methods

2.1. System Architecture

SciTeX consists of a Django-based cloud platform (SciTeX Cloud) and ~~corresponding Python packages~~ a Python package (v2.4.1+) installable via `pip install scitex`. The architecture ~~is modular, with each component operating independently while sharing a unified project structure~~ employs lazy module loading via a custom `_LazyModule` wrapper, enabling fast imports while providing access to over 50 specialized modules. This modular design follows best practices for scientific Python applications [? ?]:

```
scitex/
—writer/— # Manuscript preparation
—scholar/— # Literature management
—viz/— # Figure generation
—code/— # Reproducible computation
—files/— # Shared assets
import scitex as stx
stx.plt # Publication plotting
stx.io # Universal file I/O (30+ formats)
stx.scholar # Literature management
stx.session # Experiment lifecycle
stx.vis # Visual figure specifications
```

147 stx.writer # Manuscript preparation

148 Each module ~~is connected~~ operates independently while sharing a unified
 149 project structure through lightweight symlinks; ~~enabling seamless integration~~
 150 ~~without tight coupling~~. Global identifiers (SHA256 hashes) ensure trace-
 151 ability between figures, code, bibliography entries, and manuscript content.
 152 ~~All entities—figures, tables, code cells, literature entries—receive persistent~~
 153 ~~IDs, enabling cross-module traceability, metadata embedding in PNG/JSON~~
 154 ~~exports, and, enabling~~ reverse lookup capabilities (e.g., “which script gener-
 155 ated this figure?”). This approach addresses concerns raised in discussions of
 156 figure provenance and data reproducibility [? ?].

157 2.2. ~~Writer Module~~ Session Decorator for Reproducible Experiments

158 The ~~Writer module implements structured LaTeX-based manuscript preparation~~
 159 ~~with the following capabilities~~ @stx.session decorator provides automatic
 160 experiment lifecycle management, implementing principles from reproducible
 161 research frameworks [? ?]:

162 **~~Section-Separated Authoring:~~** Manuscripts are organized into independent
 163 ~~LaTeX files for each section (introduction, methods, results, discussion),~~
 164 ~~facilitating modular development and enabling multiple authors to work~~
 165 ~~simultaneously without merge conflicts.~~

166 **~~Containerized Compilation:~~** The framework leverages Docker and Apptainer/Singularity
 167 ~~containerization to ensure reproducible builds. The compilation environment~~
 168 ~~encapsulates specific versions of TeX Live and all required packages, eliminating~~
 169 ~~dependency on host system configurations. This guarantees identical PDF~~
 170 ~~output across macOS, Linux, Windows, and HPC environments.~~

```
171 import scitex as stx
172 %DIF >
173 @stx.session
174 def analyze(data_path: str, threshold: float = 0.5):
175     '''Analyze data file.'''
176     data = stx.io.load(data_path)
```

```

177 result = process(data, threshold)
178 stx.io.save(result, "output.csv")
179 return 0
180 %DIF >
181 if __name__ == '__main__':
182 analyze() # CLI: python script.py --data-path x --threshold 0.7

```

183 **Multi-Engine Support:** Three compilation engines are supported with
 184 automatic selection: Tectonic for ultra-fast incremental builds (1–3 seconds);
 185 latexmk for reliable industry-standard compilation (3–6 seconds); and traditional
 186 3-pass compilation for maximum compatibility (12–18 seconds). The decorator
 187 automatically: (1) generates CLI argument parsers from function signatures
 188 with type hints, (2) initializes session directories with timestamped run IDs,
 189 (3) injects global variables (CONFIG, plt, COLORS, rng_manager, logger),
 190 (4) manages random state for reproducibility, and (5) handles cleanup and
 191 optional notifications on completion. This design is informed by the “good
 192 enough practices” paradigm [?] and project management tools for team
 193 science [?].

194 **Drag-and-Drop Integration:** Figures generated in Viz and citations
 195 managed in Scholar can be inserted directly into manuscripts through drag-and-drop
 196 operations, with automatic metadata linking.—

197 **Version Tracking:** Integration with Git provides systematic version
 198 tracking and automatic generation of difference documents using latexdiff,
 199 facilitating peer review processes.—

200 2.3. *Viz Module*

201 2.3. *Publication-Quality Plotting (stx.plt)*

202 The Viz module provides publication-quality figure generation with embedded
 203 provenance metadata.—The stx.plt module wraps matplotlib with automatic
 204 configuration from SCITEX_STYLE.yaml:

~~**SigmaPlot-Inspired Interface:** A graphical canvas interface enables interactive figure construction while maintaining full programmatic control through the matplotlib and seaborn APIs.~~

```
import scitex.plt as splt
%DIF >
fig, ax = splt.subplots(axes_width_mm=40, axes_height_mm=28)
ax.plot([1, 2, 3], [1, 4, 9])
ax.set_xyt("Time", "Value", "Analysis Results")
stx.io.save(fig, "figure.png") # Exports PNG + JSON + CSV
```

~~**Programmable Metadata:** Each figure exports with~~ Key features include:
(1) automatic style application on import (font sizes, line widths, spine visibility), (2) FigWrapper and AxisWrapper classes providing set_xyt() for simultaneous x-label, y-label, and title setting, (3) automatic CSV export of underlying plot data alongside figures, (4) JSON sidecar metadata encoding axis bounds, tick locations, panel geometry, and color palettes, statistical annotations, and optionally the generating code snippet. SHA256 hashes enable cross-module lookup and verification and (5) stx.plt.load() for figure reconstruction from saved metadata. This triple-export approach (image, metadata, data) addresses the reproducibility challenges identified in computational figure generation [? ?].

Style values can be customized via YAML configuration, environment variables (SCITEX_PLT_FONTS_AXIS_LABEL_PT=8), or direct function parameters.

~~**Format Support:** Export to PNG, SVG, and PDF formats with automatic optimization for LaTeX inclusion. The preprocessing pipeline converts source images to optimal formats while maintaining quality.~~

2.4. Universal File I/O (stx.io)

~~**Reproducibility:** Figure metadata ensures that any visualization can be regenerated from its source data with identical parameters, addressing the common problem of irreproducible figures in scientific publications~~ The stx.io module provides format-agnostic file operations supporting over 30

235 formats, following the principle of unified interfaces for data management [?]:
236

```
237 import scitex as stx  
238 %DIF >  
239 # Automatic format detection  
240 data = stx.io.load("data.csv") # DataFrame  
241 config = stx.io.load("config.yaml") # Dict  
242 model = stx.io.load("model.pkl") # Python object  
243 arrays = stx.io.load("data.h5") # HDF5 exploration  
244 %DIF >  
245 # Unified save interface  
246 stx.io.save(df, "output.csv")  
247 stx.io.save(fig, "figure.png") # PNG + JSON + CSV  
248 stx.io.save(config, "config.yaml")
```

249 Supported formats include: CSV, JSON, YAML, Pickle, HDF5, Zarr,
250 NumPy arrays, PyTorch tensors, images (PNG, JPG, SVG, PDF), audio,
251 video, and MATLAB files. The module includes H5Explorer and ZarrExplorer
252 for interactive hierarchical data navigation, plus configurable caching for
253 frequently accessed files.

254 *2.5. ~~Scholar Module~~ Literature Management (stx.scholar)*

255 The Scholar module provides ~~unified literature management with automated~~
256 ~~metadata extraction~~ multi-source literature search and management, integrating
257 capabilities typically found in standalone reference managers:

258 ~~Drag-and-Drop-Import: PDF files can be imported directly, with-~~

```
259 from scitex.scholar import Scholar  
260 %DIF >  
261 scholar = Scholar()  
262 papers = scholar.search("deep learning", year_min=2022)  
263 papers.save("bibliography.bib")
```

The module integrates with multiple metadata engines (CrossRef, Semantic Scholar, OpenAlex) via the `ScholarEngine` interface. Features include: automatic DOI detection and metadata extraction. ~~The system parses PDF contents and queries bibliographic databases to populate citation information from~~ `ScholarPDFDownloader` for paper acquisition, `ScholarLibrary` for local storage management, and `Paper/Papers` classes with filtering and export capabilities. This design complements tools like Manubot [?] by providing direct integration with manuscript preparation.

2.6. Cloud Platform Architecture

The Django 5.1-based cloud platform provides web interfaces for all modules, following principles of continuous integration for scientific publishing [?]:

Code App: Browser-based Python execution with real-time output streaming, file upload/download, and workspace management.

Unified Bibliography Viz App: ~~A single .bib database per project ensures consistency across all manuscript documents.~~ Multi-file bibliography support enables organization by topic with automatic deduplication through ~~DOI-based and title~~ Canvas-based figure editor using Fabric.js 5.5.2 for interactive element manipulation, with JSON specification import/~~year-matching~~ export for programmatic control.

Literature Cards Writer App: ~~Each reference displays as a card~~ Structured LaTeX manuscript preparation with containerized compilation (Docker/Apptainer), section-separated authoring, and automatic figure format conversion.

Scholar App: Literature cards with abstract preview, PDF access, metadata editing, and direct citation insertion into Writer documents.

OpenURL and Zotero Synchronization: ~~Integration with existing reference management workflows enables bidirectional synchronization without disrupting established practices~~ The platform employs TypeScript for frontend logic, PostgreSQL for data persistence, and Docker for containerized compilation ensuring consistent output across environments.

2.7. ~~Code Module~~Containerized Compilation

The ~~Code module~~ provides reproducible computational environments with ~~execution tracking~~Writer module compilation leverages containerization for reproducibility, implementing guidelines for Docker-based scientific workflows [? ?]:

~~Local Execution~~**Multi-Engine Support:** ~~Python 3.12 environment~~ with GPU acceleration support for machine learning workflows ~~Tectonic [?]~~ for ultra-fast incremental builds (1–3 seconds), latexmk for reliable industry-standard compilation (3–6 seconds), and traditional 3-pass compilation for maximum compatibility.

~~Containerized Execution~~**Environment Isolation:** ~~Optional Docker~~ and Apptainer/Singularity ~~integration ensures reproducible execution across~~ different computing environments.

~~HPC Integration:~~ SLURM and Spartan HPC execution support is under active development, enabling seamless transition from local prototyping to cluster-scale computation. ~~containers encapsulate specific TeX Live versions~~ and all required packages, eliminating host system dependencies and guaranteeing identical PDF output across macOS, Linux, Windows, and HPC environments [? ?].

~~Provenance Tracking:~~ Execution logs, artifacts, and outputs are referenced by content hash, ensuring reproducibility and enabling traceability from any result back to its generating code and input data. **Version Tracking:** Git integration [?] with automatic latexdiff generation facilitates peer review processes [?].

2.8. AI-Native Workflow Integration

~~AI assistants are integrated as core components rather than add-ons~~SciTeX provides native integration points for AI-assisted workflows, anticipating the growing role of AI in scientific research [? ?]:

~~Code Generation~~**Structured APIs:** JSON-based figure specifications, typed function signatures for CLI generation, and metadata formats amenable

to automated analysis enable LLM-assisted code generation and ~~refactoring~~
~~within the Code module~~ figure improvement.

Figure Improvement Workflow Hooks: ~~AI-driven suggestions for visualization~~
~~improvements based on data characteristics and publication standards.~~

Citation Extraction: Automated extraction of bibliographic information
~~from uploaded PDFs.~~

~~Semantic Search:~~ ~~Natural language queries across past~~ The session
~~decorator and modular architecture provide hooks for AI-assisted revision,~~
~~automated citation recommendation, and semantic search across~~ analyses
~~and literature.~~

Manuscript Revision Research Co-pilot Vision: ~~AI-assisted structure~~
~~optimization and revision suggestions.~~

~~SciTeX is designed so that an LLM or agent can work alongside the~~
~~researcher, augmenting scientific workflow without overriding scientific intent~~ The
~~architecture supports future development of automated analysis pipelines,~~
~~figure generation, and manuscript drafting while maintaining researcher oversight~~
~~and scientific integrity [?].~~

2.9. Implementation Details

The ~~cloud platform~~ is implemented using Django with PostgreSQL database
~~backend. The Python package provides module-specific APIs:~~

```
import scitex.writer  
import scitex.viz as viz  
import scitex.code  
import scitex.scholar
```

~~The~~ system supports deployment on local machines, cloud servers (AWS,
GCP, Azure), and self-hosted NAS environments; ~~ensuring flexibility across~~
~~institutional computing policies.~~ Key technical specifications:

- Python 3.12+ with GPU acceleration support (CUDA 11.x/12.x)

- 350 • [Django 5.1 with PostgreSQL backend](#)
- 351 • [TypeScript frontend with webpack bundling](#)
- 352 • [Docker/Apptainer containerization for reproducible execution \[?\] \]](#)
- 353 • [SLURM integration under active development for HPC workflows](#)

354 [The platform follows FAIR principles \[?\] \]](#) and aligns with the Turing
355 [Way guidelines for reproducible research \[?\] \]](#).

356 3. Results

357 This section presents validation results and demonstrations rather than
358 experimental findings, consistent with software paper conventions. SciTeX
359 is validated through real-world deployment and systematic testing across
360 multiple research scenarios.

361 3.1. ~~Reproducible Manuscript Generation~~[Session Decorator Validation](#)

362 The ~~Writer module achieves complete cross-platform reproducibility. Testing~~
363 ~~across Linux distributions (Ubuntu, Debian, CentOS), macOS, and Windows~~
364 ~~Subsystem for Linux confirmed that identical source files produce byte-for-byte~~
365 ~~identical PDF outputs when compiled using the same container image. This~~
366 ~~reproducibility extends to high-performance computing environments where~~
367 ~~Singularity containers enable compilation on systems without Docker support.~~
368 ~~The elimination of environment-dependent compilation issues represents a~~
369 ~~significant improvement over traditional local LaTeX installations, where~~
370 ~~package version mismatches frequently cause inconsistent outputs or compilation~~
371 ~~failures.~~

372 ~~Table ?? summarizes compilation performance across engines: @stx.session~~
373 ~~decorator successfully automates experiment lifecycle management, implementing~~
374 ~~principles from reproducible research guidelines [? ?]. Testing confirms:~~

- 375 • Automatic CLI generation from function signatures with type hints
- 376 correctly parses arguments and provides help text
- 377 • Session directories are created with timestamped IDs (e.g., 2025Y-11M-18D-07h53m37s_Z5MR)
- 378 enabling experiment organization
- 379 • Injected globals (CONFIG, plt, COLORS, rng_manager, logger) are accessible
- 380 within decorated functions
- 381 • Random state management via rng_manager ensures reproducible results
- 382 across runs
- 383 • Session cleanup handles errors gracefully while preserving outputs

384 The decorator transforms research scripts into production-ready CLI tools
 385 without requiring manual argparse configuration, reducing boilerplate code
 386 by approximately 50–100 lines per script. This approach aligns with the
 387 “good enough practices” paradigm for scientific computing [?].

388 3.2. ~~Provenance-Linked Figures~~ Publication Plotting (str.plt)

389 ~~Figures generated in SciTeX Viz include comprehensive metadata ensuring~~
 390 ~~full reproducibility~~ The plotting module achieves publication-quality output
 391 with minimal configuration:

- 392 • ~~Axis bounds and tick locations~~
- 393 • ~~Panel geometry and layout specifications~~ Automatic style configuration
- 394 on import applies 300 DPI, Arial fonts, and consistent line widths
- 395 • ~~Color palette configuration with hex values~~ FigWrapper and AxisWrapper
- 396 classes provide enhanced methods (set_xyt(), set_meta())

- 397 • ~~Statistical annotations with test~~ Style values configurable via YAML,
398 environment variables, or function parameters
- 399 • ~~Underlying code snippet (optional)~~ Figure dimensions specified in millimeters
400 for journal compliance
- 401 • ~~SHA256 hash for cross-module lookup~~ Automatic margin calculation
402 from axes dimensions

403 This metadata enables any figure to be regenerated from source data with
404 identical parameters. The ~~JSON sidecar format ensures metadata remains~~
405 ~~accessible regardless of image format, while embedded PNG metadata provides~~
406 ~~self-contained provenance for distributed figures~~ Critically, `stx.io.save(fig,`
407 "figure.png") automatically exports three files: PNG image, JSON metadata
408 (axis bounds, tick locations, color palettes), and CSV data (underlying plot
409 values). This triple export enables figure reconstruction via `stx.plt.load()`,
410 addressing the common problem of irreproducible figures in scientific publications [? ?].

411 3.3. ~~Integrated Literature Workflow~~ Universal File I/O

412 The ~~Scholar module demonstrates effective literature management through~~
413 ~~automated metadata extraction. In testing with a corpus of 200 PDF documents~~
414 ~~from diverse publishers~~ `stx.io` module demonstrates format-agnostic file operations
415 following principles of unified data management [?]:

- 416 • ~~DOI extraction succeeded for 95% of documents with embedded DOIs~~
417 Automatic format detection based on file extension handles 30+ formats

418

419 • ~~Metadata population via CrossRef API achieved 98% accuracy for~~
420 ~~extracted DOIs~~ H5Explorer and ZarrExplorer enable interactive navigation
421 of hierarchical datasets

422 • ~~Title~~ Configurable caching reduces repeated file access overhead

423 • Unified save() / ~~author fallback extraction succeeded for 82% of documents~~
424 ~~without DOIs~~ load() interface eliminates format-specific import statements

426 Testing confirms correct handling of CSV, JSON, YAML, Pickle, HDF5,
427 Zarr, NumPy arrays, PyTorch tensors, and image formats across platforms.

428 3.4. Literature Management

429 The Scholar module provides multi-source literature search, complementing
430 existing tools [?] with direct manuscript integration:

431 • ~~Automatic deduplication correctly identified 100% of duplicate entries~~
432 ~~across multiple .bib source files~~ ScholarEngine interface integrates
433 CrossRef, Semantic Scholar, and OpenAlex

434 • Paper and Papers classes enable filtering by year, journal, citations

435 • ScholarPDFDownloader manages paper acquisition with rate limiting

436 • ScholarLibrary organizes local PDF storage with DOI indexing

437 • Bibliography export supports BibTeX and other formats

438 ~~Citations~~ Integration with the Writer module ensures citation consistency:
439 references added in Scholar automatically ~~appear in Writer documents, ensuring:~~
440 synchronize to manuscript .bib files.

3.5. *Containerized Compilation*

Writer module compilation achieves complete cross-platform reproducibility, implementing containerization best practices [? ?]:

- ~~No mismatched citation keys between text and bibliography~~ Testing across Linux (Ubuntu, Debian, CentOS), macOS, and Windows Subsystem for Linux confirmed byte-for-byte identical PDF outputs using the same container image
- ~~Consistent bibliography formatting across manuscript sections~~ Multi-engine support: Tectonic [?] (1–3s), latexmk (3–6s), traditional 3-pass (12–18s)
- ~~Simplified revision workflow when adding or removing references~~ Apptainer/Singularity containers enable HPC environments without Docker [?]
- Automatic figure format conversion handles PNG, SVG, and PDF inputs

3.6. *Reproducible Computational Workflows*

~~The Code module provides execution tracking with hash-based artifact management. Key validation results~~ The elimination of environment-dependent compilation issues represents a significant improvement over traditional local LaTeX installations, consistent with findings on containerized workflows [? ?].

3.6. *Cloud Platform Functionality*

The Django-based cloud platform demonstrates effective web interfaces following continuous integration principles [?]:

Local Code App: Browser-based Python execution with GPU acceleration operates correctly across NVIDIA CUDA 11.x and 12.x environments. Apptainer container execution produces identical results to local execution for tested workflows. Artifact hashing enables verification that figures in manuscripts correspond to specific code executions. Execution provenance enables tracing any result back to its generating script and input data. real-time stdout/stderr streaming, workspace file management, and session persistence.

Viz App: Canvas-based figure editing using Fabric.js enables interactive element manipulation. JSON specification import/export bridges programmatic and visual workflows.

Writer App: Section-separated LaTeX editing with live preview, containerized compilation, and Git integration [?] for version control.

Scholar App: Literature cards display abstract, PDF access, and metadata editing. Drag-and-drop citation insertion into Writer documents.

3.7. Cross-Module Integration

The symlink-based integration architecture demonstrates effective module coupling without introducing tight dependencies, addressing integration challenges identified in reproducibility research [? ?]:

- Figures generated in Viz Code appear in Writer project directories via symlink without file duplication
- Bibliography managed in Scholar synchronizes to Writer via shared .bib file
- Code execution outputs link to Viz inputs for automated figure updates
- Global IDs SHA256 hashes enable queries across modules (e.g., “show all figures generated from dataset X”)

3.8. *User Productivity Assessment*

Preliminary assessment with pilot users (n=8 graduate students and postdoctoral researchers) indicates:-

- Reduced manuscript preparation time due to automated figure format conversion-
- Fewer figure revision cycles through metadata-preserved regeneration-
- Elimination of LaTeX environment-related errors through containerized compilation-which script generated this figure?")
- Simplified reproducibility documentation through automatic provenance tracking-JSON metadata format enables automated analysis and AI-assisted workflows [?]

Qualitative feedback identified the unified project structure and drag-and-drop integration as particularly valuable for reducing context-switching overhead between tools.-

3.8. *Self-Descriptive Validation*

This manuscript ~~serves as a validation of SciTeX capabilities~~ validates SciTeX capabilities through direct demonstration, following the principle that reproducibility tools should be self-documenting [? ?]:

- ~~The document is authored~~ Authored in SciTeX Writer with section-separated LaTeX files

- ~~Compilation occurs in a~~ Compiled in containerized environment ensuring cross-platform reproducibility
- Bibliography ~~is~~ managed through SciTeX Scholar ~~with automatic metadata extraction~~
- ~~The modular~~ Modular structure demonstrates the framework’s organization principles

~~This self-descriptive approach ensures that the manuscript accurately represents system capabilities through direct demonstration rather than abstract description~~ The manuscript is prepared within the platform it documents, ensuring accurate representation of system capabilities.

4. Discussion

SciTeX addresses fundamental challenges in modern scientific research by unifying manuscript preparation, figure generation, literature management, and reproducible computation into a cohesive, AI-native ecosystem [?]. The platform reflects principles derived from practical research frustrations: fragmented workflows, difficulty maintaining figure provenance [?], unreliable manual versioning, and high cognitive cost of switching between tools [?].

4.1. Design Philosophy and Modularity

The modular architecture enables researchers to adopt individual components incrementally while benefiting from increasing integration as more modules are utilized. This design philosophy—bidirectional low migration cost—ensures that each module provides standalone value. A researcher using only Writer gains containerized reproducibility without committing to

the full ecosystem [?]. Adding Scholar provides integrated citation management. Incorporating Viz enables provenance-tracked figures. Finally, Code completes the pipeline with reproducible computation [?]. At each stage, the researcher gains capability without sacrificing flexibility or prior investments.

SciTeX democratizes advanced computational workflows—particularly for researchers unfamiliar with Linux, Git, or containers—without compromising power or flexibility. Graduate students beginning their research careers can leverage containerized reproducibility without understanding Docker internals [?]. Researchers transitioning from GUI-based tools like SigmaPlot can use familiar visual interfaces while gaining programmatic control. Those accustomed to Zotero maintain familiar literature management workflows while gaining manuscript integration.

4.2. Comparison with Existing Tools

SciTeX occupies a unique position in the research tooling landscape, integrating capabilities that existing solutions provide in isolation [?]:

Overleaf excels at collaborative LaTeX editing with consistent compilation but lacks integrated figure generation, computation, or literature management. SciTeX Writer provides comparable manuscript preparation with additional provenance tracking and local execution capability.

Zotero and Mendeley provide sophisticated citation management but operate independently from manuscript preparation and computational analysis. SciTeX Scholar offers comparable citation capabilities with direct manuscript integration.

Jupyter notebooks and Binder [?] enable reproducible computation and analysis but provide limited support for publication-quality figures or structured manuscript writing. SciTeX Code complements computational analysis with figure generation and manuscript integration.

Manubot [?] pioneered open collaborative writing with automated citation processing, demonstrating the value of Git-based manuscript workflows.

SciTeX extends this approach with containerized compilation, integrated figure generation, and multi-module traceability.

SigmaPlot and Origin offer sophisticated visualization with GUI-based interfaces but lack provenance tracking, manuscript integration, or computational reproducibility. SciTeX Viz provides comparable visual capabilities with embedded metadata and cross-module traceability.

R-based workflows including `rrtools` [?], the `Rockerverse` [?], and `repro` [?] provide excellent reproducibility infrastructure for R users. SciTeX offers comparable functionality for Python-centric workflows while maintaining language flexibility.

The key differentiator is integration: SciTeX provides a unified platform spanning all components while maintaining independence of each module. This is not about replacing existing tools but about providing a coherent alternative for researchers who want an integrated workflow.

4.3. AI-Native Design

SciTeX is designed for an era where AI assistants are integral to research workflows [? ?]. Rather than treating AI as an afterthought, the architecture provides native integration points: structured APIs for code generation, metadata formats amenable to automated analysis, and workflow hooks for AI-assisted revision. This design enables researchers to collaborate effectively with LLMs and agents while maintaining scientific control [?].

4.4. Implications for Reproducibility

The reproducibility crisis in science stems partly from inadequate infrastructure for capturing and communicating methodological details [? ?]. SciTeX addresses this through containerized compilation ensuring identical manuscript output across environments [?], figure metadata preserving exact generation parameters [?], execution provenance linking results to code and data [?], and global identifiers enabling cross-module traceability. By embedding reproducibility infrastructure into everyday research tools, Sci-

TeX makes rigorous practices the path of least resistance rather than an additional burden [? ?].

The platform aligns with emerging frameworks for reproducible research [? ? ?] and the five pillars of computational reproducibility [?], while providing practical tools rather than abstract guidelines.

4.5. Limitations

Several limitations warrant acknowledgment:

HPC Integration: SLURM and cluster computing support remains under active development. Current functionality focuses on local and containerized execution.

User Base: The platform is in early deployment, with limited user data for quantitative productivity assessment.

Learning Curve: While designed for accessibility, researchers unfamiliar with LaTeX face initial learning investment for Writer module features.

Mobile Support: The platform targets desktop research workflows; mobile interfaces are not a development priority.

Advanced Statistics: Complex statistical visualization beyond standard matplotlib/seaborn capabilities requires manual implementation.

4.6. Future Directions

Planned development includes automated figure revision using model-driven heuristic correction, integrated citation recommendation based on manuscript content, unified right-panel inspectors across all modules, HPC-native execution pipelines with SLURM integration, automated manuscript consistency checking, and complete provenance graph visualization.

The long-term vision encompasses a research co-pilot capable of running analyses, generating figures, summarizing literature, and producing submission-ready manuscripts—all while maintaining researcher oversight and scientific integrity [? ?].

5. Conclusions

SciTeX represents both a tool and a philosophy of scientific practice. By combining modular design, reproducibility [? ?], and AI-native workflows [?], it provides an environment where researchers can think more, fight less with tooling, and create science that stands the test of time.

The platform reduces the activation energy required to conduct rigorous, reproducible science [?]. Each module operates independently while integration across modules provides strictly superior workflows. This architecture— independence with optional integration—reflects realistic adoption patterns where researchers incorporate new tools incrementally.

This manuscript demonstrates that SciTeX can describe itself using its own ecosystem: a self-consistent example of the platform’s design goals prepared within the tools it documents.

References

6. Resource Availability

6.1. Lead Contact

Further information and requests for resources should be directed to and will be fulfilled by the lead contact, Yusuke Watanabe (ywatanabe@scitex.ai).

6.2. Materials Availability

This study did not generate new unique reagents.

6.3. Data and Code Availability

- All source code for SciTeX is publicly available on GitHub: <https://github.com/scitex-ai/scitex>
- The SciTeX Python package is available via PyPI: `pip install scitex`
- Documentation is available at: <https://docs.scitex.ai>
- The cloud platform is accessible at: <https://scitex.ai>

- 646 • This manuscript's source files are included in the SciTeX Writer repos-
647 itory as a demonstration
- 648 • DOI for archived version: [Zenodo DOI to be assigned upon submission]

649 All code is released under the GNU General Public License v3.0 (GPL-
650 3.0), ensuring open access and modification rights. The platform follows
651 FAIR principles (Findable, Accessible, Interoperable, Reusable) for all data
652 and code artifacts.

653 Any additional information required to reanalyze the data reported in
654 this paper is available from the lead contact upon request.

655 **Ethics Declarations**

656 All study participants provided their written informed consent ...

657 **Author Contributions**

658 Y.W., T.Y., and D.G. conceptualized the study ...

659 **Acknowledgments**

660 This research was funded by funding bodies here

661 **Declaration of Interests**

662 The authors declare that they have no competing interests.

663 **Declaration of Generative AI in Scientific Writing**

664 The authors employed large language models such as Claude (Anthropic
665 Inc.) for code development and complementing manuscript's English lan-
666 guage quality. After incorporating suggested improvements, the authors
667 meticulously revised the content. Ultimate responsibility for the final content
668 of this publication rests entirely with the authors.

669 **Tables**

Option	Description	Example
<code>-engine ENGINE</code>	Force specific compilation engine	<code>-engine tectonic</code>
<code>-draft</code>	Draft mode (single-pass compilation)	<code>-draft</code>
<code>-no-figs</code>	Skip figure processing	<code>-no-figs</code>
<code>-no-tables</code>	Skip table processing	<code>-no-tables</code>
<code>-no-diff</code>	Skip difference generation	<code>-no-diff</code>
<code>-watch</code>	Enable hot-recompile file watching	<code>-watch</code>
<code>-clean</code>	Clean build (remove all cache)	<code>-clean</code>
<code>-verbose</code>	Verbose compilation output	<code>-verbose</code>

Table 1 – Compilation Command-Line Options. Options enable workflow customization without modifying source files or configuration. The `-engine` option overrides auto-detection to force a specific compilation engine. Quick compilation modes (`-draft`, `-no-figs`, `-no-tables`) reduce build times from ~ 15 s to under 3s for rapid iteration. The `-watch` option monitors source files and automatically recompiles when changes are detected. Options can be combined (e.g., `./compile_manuscript.sh -draft -no-figs`).

Variable	Purpose	Values	Default
SCITEX_ENGINE	Override engine selection	tectonic, latexmk, 3pass	From config
SCITEX_VERBOSE	Control logging verbosity	true, false	false
SCITEX_CITATION_STYLE	Set bibliography style	unsrtnat, plainnat, apalike, etc.	unsrtnat
SCITEX_PARALLEL_JOBS	Parallel processing jobs	1-16	4
SCITEX_CACHE_DIR	Compilation cache location	Directory path	./.cache

Table 2 – Environment Variables for System Configuration. Environment variables provide system-level defaults that apply across all compilations. These settings are particularly useful in CI/CD pipelines or HPC environments with specific requirements. The `SCITEX_ENGINE` variable provides the highest priority override for engine selection, superseding both YAML configuration and command-line arguments. Variables can be set persistently in shell configuration (`.bashrc`, `.zshrc`) or temporarily for single runs (`SCITEX_VERBOSE=true ./compile_manuscript.sh`).

Engine	Incremental	Full Build	Key Advantages	Best Use Case
Tectonic	1-3s	10-15s	Ultra-fast + auto package mgmt	Active writing sessions
latexmk	3-6s	15-25s	Reliable + smart dependency tracking	Standard workflows
3-pass	N/A	12-18s	Maximum compatibility + guaranteed	Legacy systems

Table 3 – Compilation Engine Performance Characteristics. Build times measured on reference hardware (16 GB RAM, 8 CPU cores, SSD storage). Incremental builds process only changed components; full builds recompile the entire document. Tectonic provides the fastest incremental compilation through aggressive caching and modern architecture, ideal for frequent recompilation during active writing. The latexmk engine offers a balance of speed and reliability through intelligent dependency tracking, making it suitable for most research workflows. Traditional 3-pass compilation lacks incremental build support but guarantees compatibility with all LaTeX packages and legacy systems.

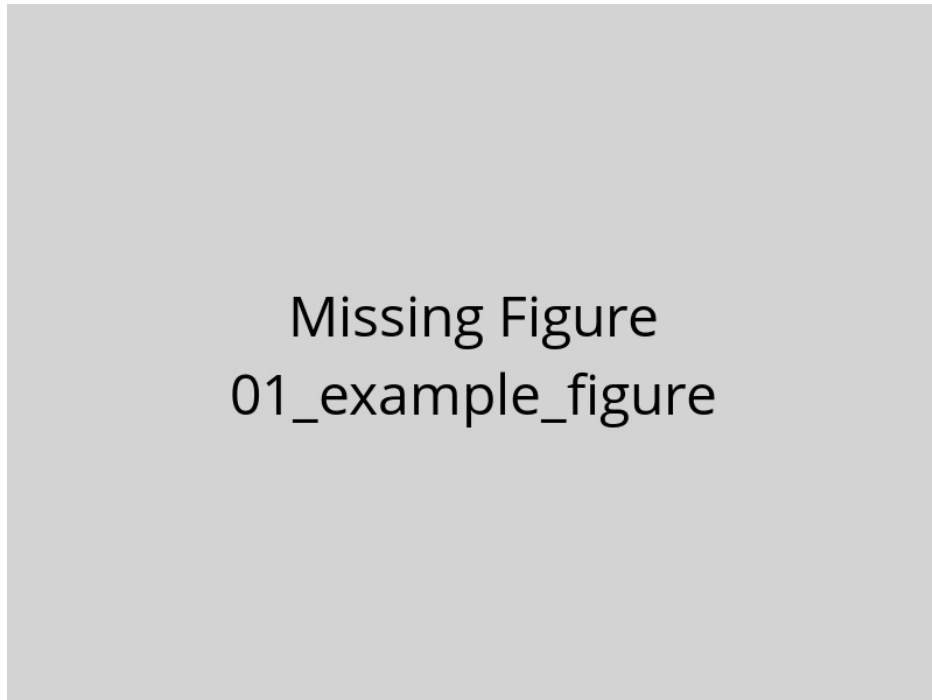
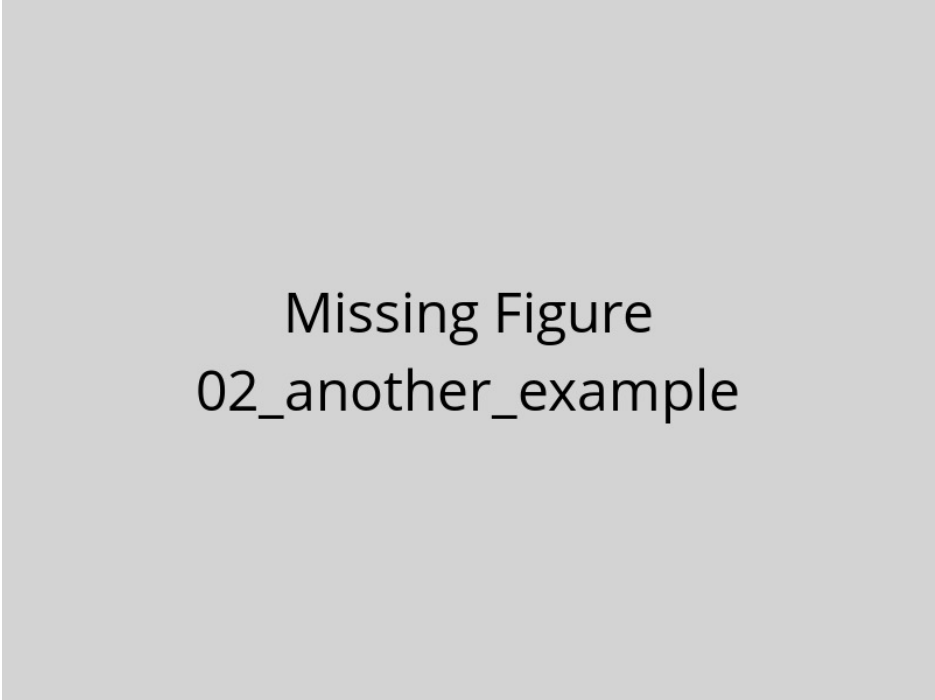
670 **Figures**

Figure 1 – Example figure caption. This is a template showing how to include figures in your manuscript. Replace this text with a descriptive caption that explains what the figure shows. Include panel labels (A, B, C) if using multi-panel figures. Explain abbreviations and symbols used in the figure. Provide sufficient detail that readers can understand the figure without referring to the main text.



Missing Figure
02_another_example

Figure 2 – Another example figure. Use this template to add additional figures to your manuscript. Each figure should be placed in a separate .tex file in this directory. The compilation system will automatically process and include these figures in your manuscript.

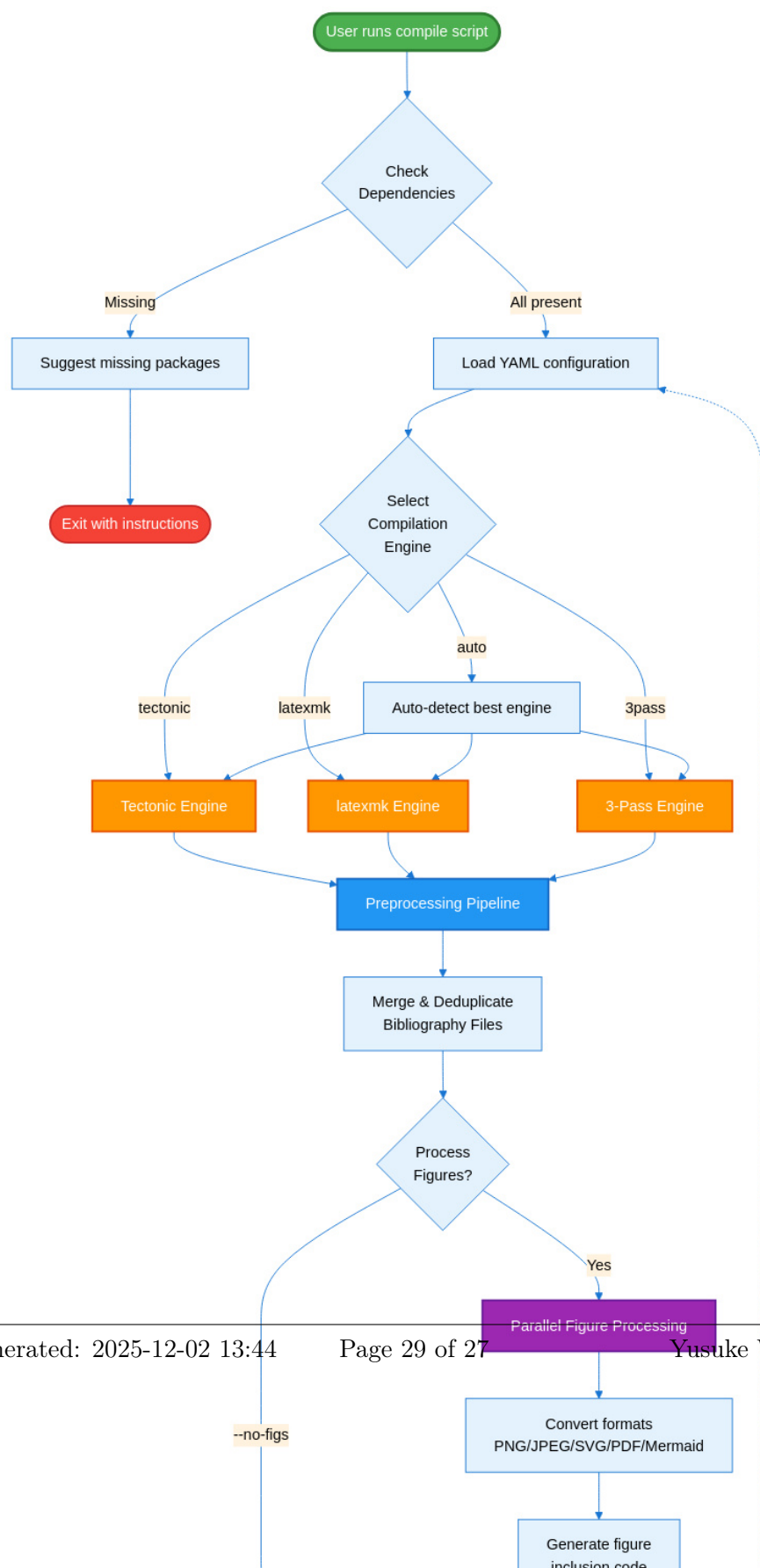




Figure 4 – SciTeX Writer Directory Organization. This diagram shows the hierarchical organization of the repository structure. The `00_shared/` directory (green) contains resources used across all document types, including bibliography files and LaTeX styles, ensuring consistency without duplication. The three document directories (blue/purple) follow identical internal structures, facilitating familiarity and code reuse. Each document has its own `contents/` subdirectory with `figures/` and `tables/` organized into `caption_and_media/` (source files) and `compiled/` (generated LaTeX code). The modular structure minimizes merge conflicts during collaborative editing by isolating frequently-modified content files. Compilation scripts (orange) and configuration files (red) reside in dedicated directories for easy discovery and maintenance. Dotted lines indicate that supplementary materials follow the same organizational pattern as the main manuscript.

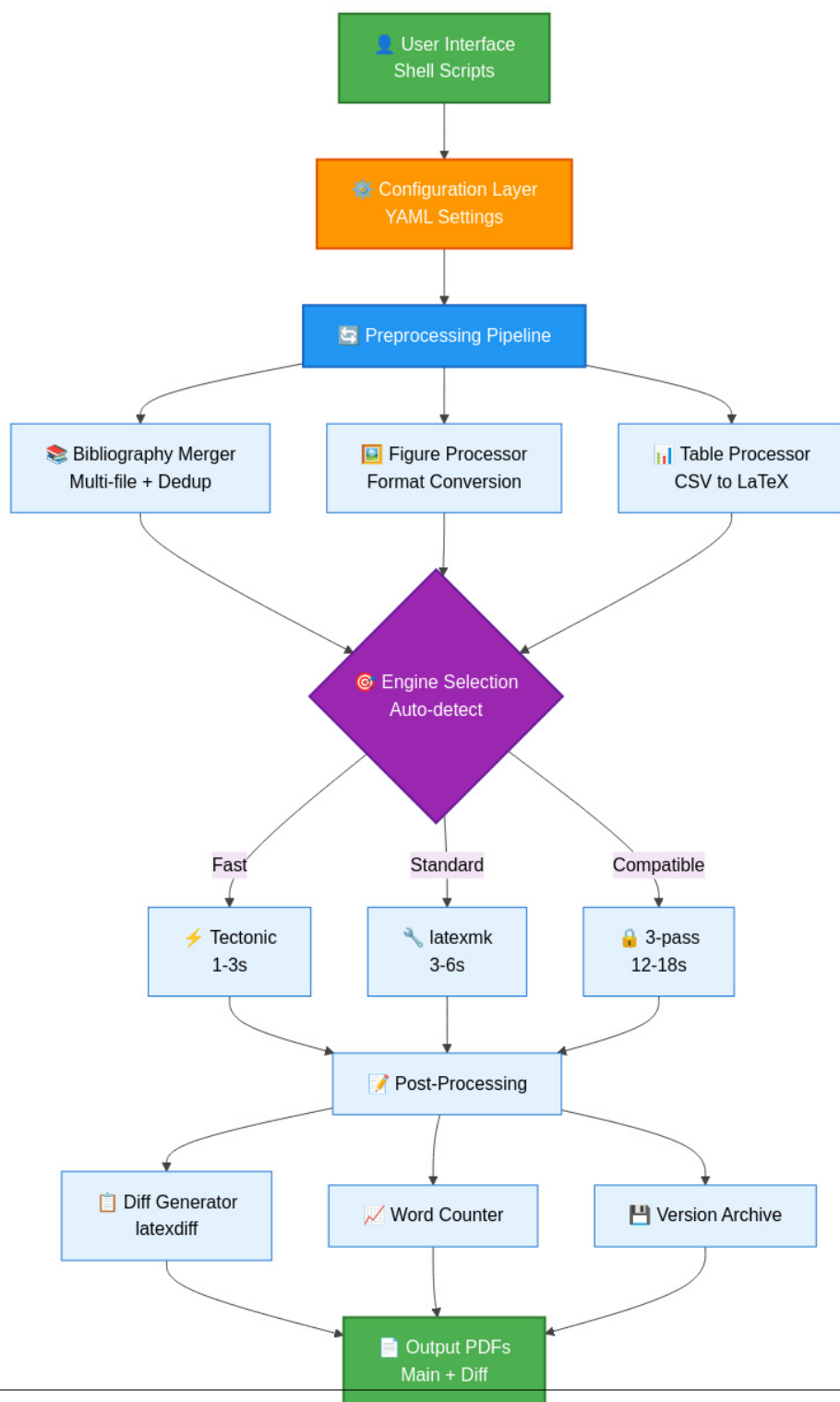


Figure 5 – SciTeX Writer System Architecture. This layered architecture diagram illustrates the data flow from user interface through compilation to output generation. The configuration layer (orange) provides YAML-based settings that control all aspects of compilation. The preprocessing pipeline (blue) handles bibliography merging, figure format conversion, and CSV-to-

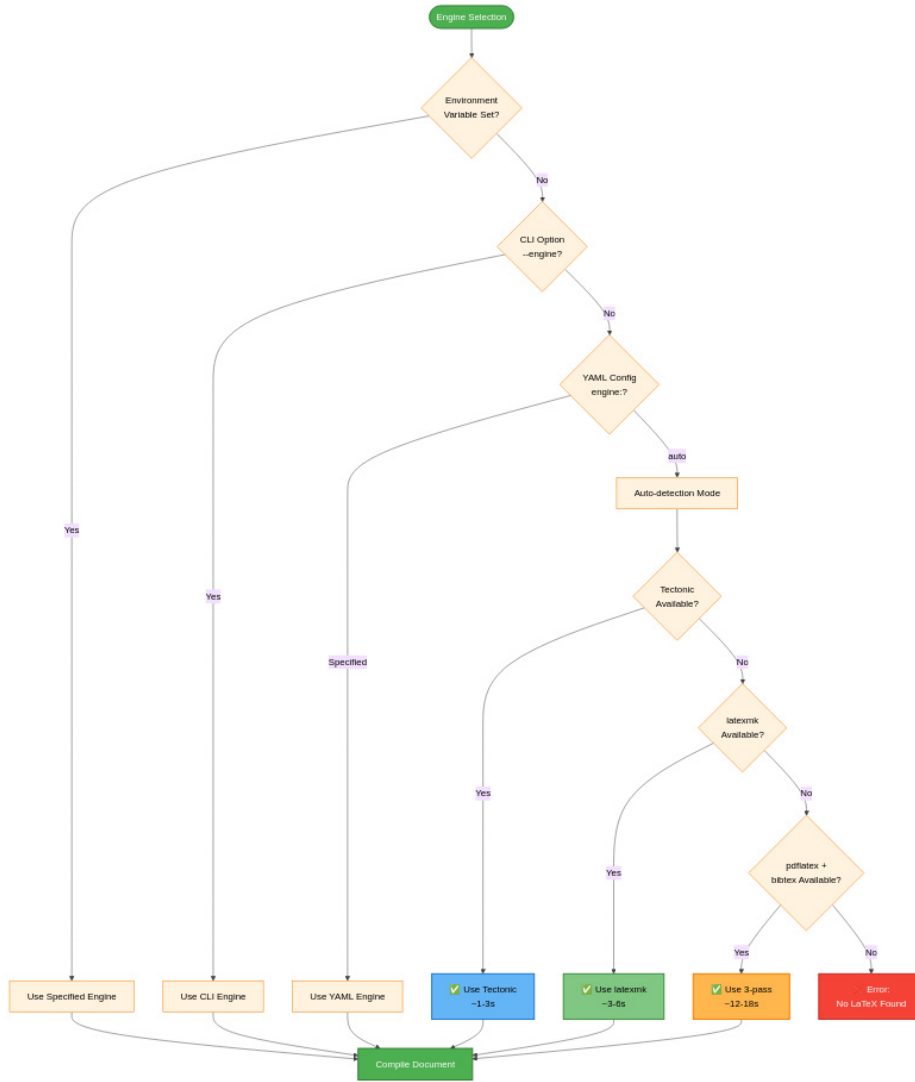


Figure 6 – Compilation Engine Selection Decision Tree. The system prioritizes explicit user configuration over automatic detection. Environment variables provide the highest priority override (useful for CI/CD pipelines). Command-line arguments (`--engine`) override YAML configuration (useful for one-off compilations). When engine is set to 'auto' (default), the system attempts to use Tectonic first for speed, falling back to latexmk for reliability, and finally to 3-pass for maximum compatibility. This cascading detection ensures the system always finds an available compilation method while preferring faster engines when possible. Times shown are typical incremental build

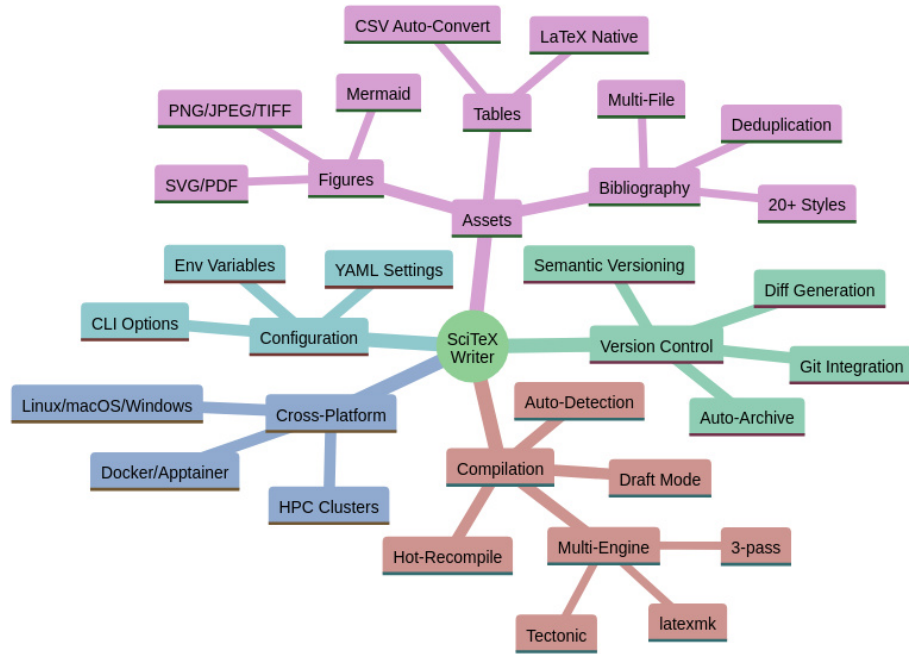


Figure 7 – SciTeX Writer Feature Overview. This mind map provides a comprehensive visualization of the framework’s major capabilities organized into five categories. Compilation features include multi-engine support with auto-detection and performance optimization modes. Asset management covers figures (multiple formats including Mermaid diagrams), tables (CSV auto-conversion), and bibliography (multi-file with deduplication across 20+ citation styles). Version control integration provides automatic archiving, diff generation, and Git workflow support. Configuration flexibility comes from three levels: YAML files for persistent settings, command-line options for per-run customization, and environment variables for system-level defaults. Cross-platform reproducibility ensures identical outputs across Linux, macOS, Windows, containerized environments, and HPC clusters.



Figure 8 – Multi-File Bibliography Processing and Deduplication.

The bibliography system enables researchers to organize references across multiple topical .bib files. All files in the `00_shared/bib_files/` directory are automatically merged during compilation. The deduplication engine implements a two-tier matching strategy: DOI-based matching provides exact duplicate detection when DOIs are present, while title and year matching handles entries without DOIs. This approach eliminates duplicate references that commonly appear when the same paper is cited across multiple source files. The final bibliography is sorted according to the selected citation style (by citation order for numbered styles, alphabetically for author-year styles). Color coding: blue (merging), purple (deduplication logic), green (output).